

JAVA 8 MASTER HANDBOOK

Lecture 1 – Java 8 Introduction

Part 3 – Java 8 Features Overview (Foundation for the Entire Course)

Goal of Part 3

Is part ke end tak tum Java 8 ke har major feature ka purpose samajh jaoge. Ye poore course ka roadmap aur conceptual foundation hai.

Table of Contents

1. Why Java 8 Introduced New Features
2. Complete List of Java 8 Features
3. Lambda Expressions (Overview)
4. Functional Interfaces (Overview)
5. Method References (Overview)
6. Stream API (Overview)
7. Default Methods
8. Static Methods in Interfaces
9. Optional
10. Date & Time API
11. Nashorn JavaScript Engine
12. Java 8 Feature Flow
13. Feature Comparison
14. Interview Questions
15. Revision Notes
16. Homework

Chapter 1 - Why Java 8 Introduced New Features?

Ab ek important question.

Java developers ne itne saare naye features ek saath kyun introduce kiye?

Answer

Industry ki requirements change ho rahi thi.

Companies ko chahiye tha:

- Less code
- Better readability
- Better maintainability
- Easier testing
- Better performance
- Parallel processing support
- Modern APIs

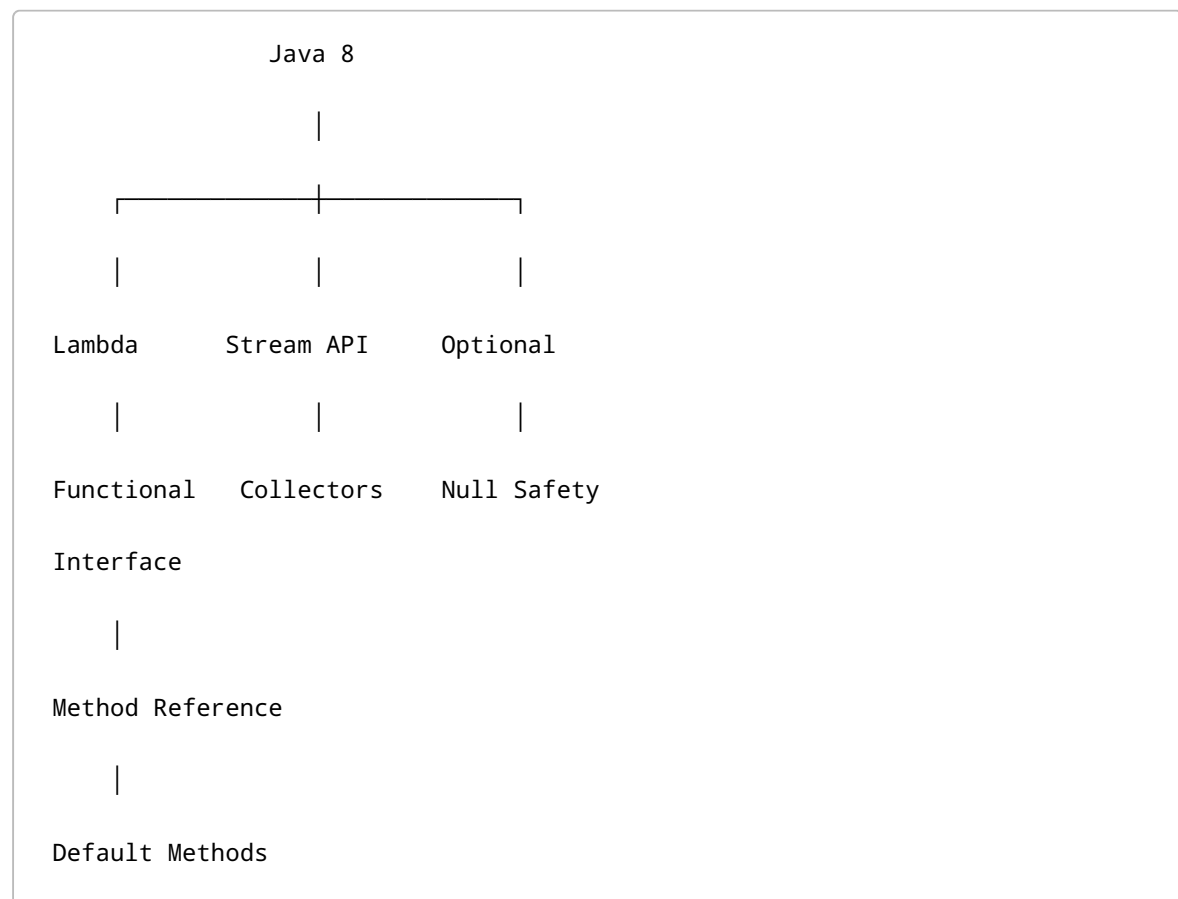
Java 7 in sab requirements ko efficiently address nahi kar pa raha tha.

Isliye Java 8 me multiple related features introduce kiye gaye.

Ye saare features alag-alag nahi hain.

Ye ek ecosystem ka hissa hain.

Java 8 Feature Ecosystem



Important Point

Ye saare features ek doosre se connected hain.

Example

Lambda

↓

Functional Interface

↓

Stream API

↓

Collectors

Agar Lambda nahi samjhe,

to Streams bhi properly nahi samajh paoge.

Chapter 2 - Complete List of Java 8 Features

Java 8 ke major features:

Feature	Importance
Lambda Expressions	★★★★★★
Functional Interfaces	★★★★★★
Stream API	★★★★★★
Method References	★★★★★
Default Methods	★★★★★
Static Methods in Interfaces	★★★
Optional	★★★★★★
Date & Time API	★★★★★★

Feature	Importance
Nashorn JavaScript Engine	★★

Interview Tip

Top 5 sabse important features:

1. Lambda Expressions
2. Functional Interfaces
3. Stream API
4. Optional
5. Date & Time API

Agar kisi interview me sirf "Java 8 ke important features batao" pucha jaye, to in paanch features ko confidently explain kar paana chahiye.

Chapter 3 - Lambda Expressions (Overview)

Problem Before Java 8

Anonymous Classes bahut verbose thi.

Example:

```
Collections.sort(list, new Comparator<Employee>() {  
    @Override  
    public int compare(Employee a, Employee b) {  
        return a.getSalary() - b.getSalary();  
    }  
});
```

Sirf compare karna tha.

Lekin itna bada code likhna pada.

Java 8 Solution

```
list.sort((a, b) -> a.getSalary() - b.getSalary());
```

Benefits:

- Code chhota ho gaya.
- Readability improve hui.
- Boilerplate code kam hua.
- Behavior ko easily pass kiya ja sakta hai.

Definition

Lambda Expression ek anonymous function-like syntax hai jo Functional Interface ke abstract method ki implementation ko concise tarike se represent karta hai.

Important Note: Java me Lambda actual function nahi hota. Ye Functional Interface ka implementation represent karta hai. Is concept ko hum Lecture 2 me internally samjhenge.

Purpose of Lambda Expressions

- Boilerplate code reduce karna.
- Behavior pass karna.
- Functional Programming support karna.
- Streams ko expressive banana.

Chapter 4 - Functional Interfaces (Overview)

Question:

Lambda kis type ka object represent karta hai?

Answer

Functional Interface ka.

Definition

Functional Interface wo interface hota hai jisme exactly **one abstract method** hota hai.

Example:

```
@FunctionalInterface
interface Calculator {

    int add(int a, int b);
}
```

```
}
```

Usage:

```
Calculator calculator = (a, b) -> a + b;
```

Yahan Lambda `Calculator` interface ke abstract method ki implementation provide kar raha hai.

Relationship

Lambda

↓

Implements

↓

Functional Interface

Ye relationship Java 8 ka foundation hai.

Quick Revision

Anonymous Class

↓

Verbose

↓

Java 8

↓

Lambda Expression

↓

Functional Interface

↓

Cleaner Code